

DEVOPS ★ CI/CD ★ PRODUCTION ENGINEERING

CI/CD Production Interview Handbook

25+ Real Production Interview Questions & Answers

PART 1 ● QUESTIONS 1-15 ● HINGLISH INTERVIEW PREP

How to Use This Handbook

This handbook feels like a real interview conversation, not a textbook. Every answer explains the thinking behind it, includes practical, production-grade reasoning, and closes with a production insight – the exact qualities interviewers look for in a DevOps engineer.

Each question is broken down the way it would actually come up in an interview: the question itself, how to answer it with confidence, common follow-up questions, and a production tip drawn from real experience.

What's inside

- **Questions 1-5** – Core CI/CD pipeline, security, and design fundamentals.
- **Questions 6-10** – Bonus questions not covered in the original reel.
- **Questions 11-15** – Deep-dive, command-level follow-ups with real `kubectl` and `docker` troubleshooting.

Commented on the reel to get this? Save this PDF – these are the exact questions being asked in real production CI/CD interviews right now.

Contents

CORE QUESTIONS	3
1. Explain Your Complete CI/CD Pipeline: Git Push to Production	3
2. Quality & Security Gates Before Deployment	4
3. Designing a Reusable Multi-Environment Pipeline	5
4. Securing Secrets, API Keys & Connection Strings	5
5. Debugging a Failed Production Deployment	6
BONUS QUESTIONS	7
Question 6	7
Question 7	7
Question 8	7
Question 9	7
Question 10	8
DEEP-DIVE QUESTIONS	9
11. Old Version in Production - A Command-Level Debug	9
12. Blue-Green vs Canary - Choosing the Right Strategy	9
13. A Failed SonarQube Quality Gate - Do You Deploy Anyway?	10
14. Pipeline Suddenly Fails - Where Do You Start?	11
15. Docker Image Builds, But the Registry Push Fails	12

CORE QUESTIONS

The five questions every CI/CD interview starts with

1

Explain Your Complete CI/CD Pipeline: Git Push to Production

INTERVIEW QUESTION

Git Push se lekar Production Deployment tak apna complete CI/CD pipeline explain karo.

How I'd Answer This

Whenever this question comes, I don't start by listing Jenkins, GitHub Actions, or Azure DevOps. Instead, I explain the flow, because interviewers want to know whether I actually understand what happens after a developer pushes code.

Let's say it's a Java Spring Boot microservice. A developer pushes code to GitHub, which immediately triggers our GitHub Actions workflow. The pipeline first checks out the latest code and installs Java. Next, Maven builds the application and runs all unit tests – if unit tests fail, the pipeline stops immediately, because there's no point building or deploying broken code.

Once the build succeeds, SonarQube runs and checks bugs, vulnerabilities, code smells, and code coverage. If the Quality Gate passes, I run dependency scanning using Snyk (or OWASP Dependency Check). After that, the Docker image is built. Before pushing the image to Azure Container Registry, I scan it using Trivy to ensure there are no critical vulnerabilities. If everything looks good, the image is pushed to ACR.

The CD pipeline then deploys the application to the Dev environment. After smoke testing, it moves to QA. Production deployment requires manual approval from the release manager. Once approved, Helm upgrades the application on AKS. Finally, smoke tests and monitoring validate that the deployment was successful.

Why this order?

Every stage answers one question:

- Can the code build?
- Does it work?
- Is the code quality acceptable?
- Is it secure?
- Can it be containerized?

- Can it be deployed safely?

PRODUCTION TIP

One mistake I often see is people saying “Build → Deploy.” A production pipeline is much more than that – quality, security, and validation are equally important.

2**Quality & Security Gates Before Deployment****INTERVIEW QUESTION**

Deployment se pehle pipeline mein kaun-kaun se Quality aur Security Gates lagate ho? Aur kaunse tools use karte ho?

How I'd Answer This

I don't rely on just one scan – different stages validate different things. Right after the build, I run unit tests to make sure the application behaves correctly. Then SonarQube checks code quality and enforces our Quality Gate. Next comes dependency scanning to identify vulnerable third-party libraries.

Once the Docker image is built, I scan it using Trivy to detect OS-level and package vulnerabilities. If any critical vulnerability is found, I fail the pipeline immediately instead of deploying it – this ensures insecure code never reaches Production.

Tools I Usually Mention

- SonarQube
- Trivy
- Snyk
- GitLeaks
- OWASP Dependency Check

FOLLOW-UP QUESTION**What if SonarQube fails?**

I don't bypass it. First I understand whether it's a genuine issue or a false positive. If it's a genuine issue, I fix the code and rerun the pipeline.

3

Designing a Reusable Multi-Environment Pipeline

INTERVIEW QUESTION

Ek reusable pipeline kaise design karoge jo Dev, QA aur Production teeno environments mein chal sake?

How I'd Answer This

I avoid maintaining three separate pipeline files. Instead, I create one reusable pipeline and pass environment-specific values using parameters or variables.

For example, only these things change between environments:

- Subscription
- Resource Group
- Namespace
- Variable Group
- Service Connection

The pipeline logic remains exactly the same. This reduces duplication and makes maintenance much easier.

PRODUCTION TIP

If you're copying the same YAML three times, your pipeline probably isn't reusable.

4

Securing Secrets, API Keys & Connection Strings

INTERVIEW QUESTION

Production passwords, API Keys aur Connection Strings pipeline mein securely kaise store aur use karte ho?

How I'd Answer This

I never hardcode secrets inside YAML files or Git repositories. Instead, I store them in Azure Key Vault. The pipeline authenticates using Managed Identity or a Service Connection, and during execution, secrets are fetched securely at runtime. This way, developers never see production passwords and secrets aren't exposed in Git history.

THINGS I NEVER DO

- ✗ Store passwords in Git
- ✗ Hardcode secrets in YAML
- ✗ Build secrets into Docker images

FOLLOW-UP QUESTION**Can GitHub Secrets replace Azure Key Vault?**

Yes, for small projects. But in enterprise environments, Azure Key Vault is generally preferred because it offers centralized secret management, rotation, RBAC, and auditing.

5**Debugging a Failed Production Deployment****INTERVIEW QUESTION**

Production deployment pipeline fail ho gayi. Tumhare first 5 debugging steps kya honge?

How I'd Answer This

I don't restart the pipeline immediately. First, I identify where exactly it failed. My debugging flow is:

1. Check pipeline logs and identify the failed stage.
2. Verify whether the Docker image was built and pushed successfully.
3. Check deployment logs and rollout status.
4. Verify Kubernetes events and application logs.
5. Identify the root cause before rerunning the deployment.

This approach saves time and prevents repeated failed deployments.

PRODUCTION TIP

A green build doesn't always mean a successful deployment. Always verify the deployment separately.

BONUS QUESTIONS

Not covered in the original reel

Question 6

Pipeline successful hai, lekin Production mein old version chal raha hai. Kya check karoge?

The first thing I'd verify is whether the latest Docker image was actually pushed. Then I'd check:

- Image tag
- Deployment image
- Rollout status
- Running Pods
- Browser/CDN cache

Many times the pipeline succeeds, but Kubernetes is still using the previous image.

Question 7

Production deployment ke liye Blue-Green aur Canary mein kya choose karoge?

It depends on the application. If I need instant rollback with almost zero downtime, I'd choose Blue-Green. If I want to release gradually and monitor real user traffic before a full rollout, I'd choose Canary.

Question 8

Pipeline mein ek hi job ko multiple Java versions pe kaise run karoge?

In GitHub Actions, I'd use `strategy.matrix`. This allows me to test the same application against Java 17 and Java 21 in parallel, reducing overall execution time.

Question 9

Agar SonarQube Quality Gate fail ho jaye toh kya karoge?

I won't bypass the Quality Gate. I'll review the report, fix the issues if they're valid, and rerun the pipeline. Quality Gates are there to prevent poor-quality code from reaching production.

Question 10***Apne career ka ek real CI/CD issue batao.***

One issue I faced was that the pipeline completed successfully, but the application was still running the old version. After checking the rollout, I found that the Deployment was still using the `latest` image tag and the image wasn't being pulled again. We switched to immutable version tags based on the build number, updated the deployment, and the issue was resolved. After that, we completely stopped using `latest` in production.

DEEP-DIVE QUESTIONS

Command-level follow-ups with real troubleshooting

11 Old Version in Production – A Command-Level Debug

INTERVIEW QUESTION

Pipeline successful tha, lekin Production mein old version hi chal raha tha. Kaise debug karoge?

How I'd Answer This

The first thing I check is whether the latest Docker image was actually built and pushed. Then I verify whether the deployment is using the correct image tag. If the deployment still points to an old image or uses the `latest` tag, Kubernetes may not pull the new image. Next I check the rollout status, then verify the image running inside the Pod:

```
# check rollout status
kubectl rollout status deployment <deployment-name>

# verify the image running inside the pod
kubectl describe pod <pod-name>
```

If everything looks correct, I move to the Service and Ingress to ensure traffic is reaching the latest Pods. In a few cases, browser cache or CDN cache can also make users see the old version.

PRODUCTION TIP

Never use `latest` in Production. Always use immutable image tags like Build ID or Git Commit SHA.

12 Blue-Green vs Canary – Choosing the Right Strategy

INTERVIEW QUESTION

Production deployment ke liye Blue-Green aur Canary mein kya choose karoge?

How I'd Answer This

It depends on the application. If the application is business critical and I need instant rollback, I'd choose Blue-Green Deployment, because both environments remain ready – if any issue occurs, traffic can immediately switch back.

If I want to reduce deployment risk and observe production traffic gradually, I'd use Canary Deployment, rolling traffic forward in stages:

5% → 20% → 50% → 100%

This allows me to monitor metrics before rolling out to everyone.

FOLLOW-UP QUESTION**Which one do you use more?**

Mostly Rolling Update. But for critical services, Blue-Green or Canary is much safer.

13 A Failed SonarQube Quality Gate – Do You Deploy Anyway?

INTERVIEW QUESTION

SonarQube Quality Gate fail ho gaya. Kya deployment continue karoge?

How I'd Answer This

No. I first open the SonarQube report and verify whether the issue is a bug, a vulnerability, a code smell, or low code coverage:

- Bug
- Vulnerability
- Code Smell
- Low Code Coverage

If it's a genuine issue, I fix the code, commit again, and the pipeline reruns automatically. Only after the Quality Gate passes do I allow deployment.

FOLLOW-UP QUESTION**What if it's a false positive?**

I verify it with the development team. If justified, I can mark it appropriately, but I never ignore a Quality Gate without understanding the reason.

PRODUCTION TIP

Quality Gate should block bad code, not become a checkbox.

14**Pipeline Suddenly Fails – Where Do You Start?****INTERVIEW QUESTION**

Pipeline suddenly fail hone lagi, lekin kal tak sab sahi chal raha tha. Kahan se debugging start karoge?

How I'd Answer This

I never assume the code is the problem. My first step is identifying what changed since the last successful run. Then I check:

- Pipeline logs
- Recent commits
- Runner health
- Service Connection
- Secret expiration
- Registry connectivity
- Cloud outage

Many times, the issue isn't in the application – it could be an expired Service Principal, an Azure outage, or even a GitHub Runner issue.

PRODUCTION TIP

Always compare the last successful pipeline with the failed one. That usually saves a lot of debugging time.

15

Docker Image Builds, But the Registry Push Fails

INTERVIEW QUESTION

Pipeline mein Docker image successfully build ho gayi, lekin Registry push fail ho gaya. Kaise troubleshoot karoge?

How I'd Answer This

First I verify the exact error message. Then I check:

- Registry credentials
- Service Connection
- Login command
- Repository permissions
- Network connectivity
- Image tag

If authentication succeeds, I'll verify whether the repository exists. Sometimes the issue is simply a missing permission on Azure Container Registry.

```
# authenticate to the registry
docker login

# push the image
docker push
```

PRODUCTION TIP

Whenever possible, use Managed Identity or Federated Identity instead of storing registry credentials inside the pipeline.

End of Part 1

Questions 1–15 of the CI/CD Production Interview Handbook

More real production interview questions are coming in the next drop – follow along on the reel.